Complete IDP Documentation

Beginner understanding -> actual implementation -> security -> production IAM

Source: inspected repository at C:/Users/itzme/Desktop/All Stacks/03-IDP
The supplied prompt contains 64 numbered topics; this PDF preserves all 64.

Contents

Each numbered topic begins on a new page.

1	Analyze the Entire Project First
2	Project Overview
3	What Is an Identity Provider?
4	Why Do We Need an IDP?
5	Complete IDP Architecture
6	Technology Stack
7	Complete User Authentication Flow
8	User Registration Flow
9	Password Security
10	Login Flow
11	Login API Documentation
12	Token Explanation
13	JWT Explanation
14	JWT Creation Flow
15	JWT Validation Flow
16	Cookies vs Local Storage vs Memory
17	Authentication vs Authorization
18	Roles
19	Permissions
20	Protected Route Flow
21	Complete API Documentation
22	Authentication API Categories
23	Logout Flow
24	Refresh Token Flow
25	Session-Based Authentication
26	OAuth 2.0
27	OpenID Connect
28	ID Token vs Access Token
29	Complete Request to Response Flow
30	Error Handling
31	Security Analysis
32	Environment Variables
33	Database Architecture
34	Database Flow
35	IDP vs Application
36	Single Sign-On
37	Real-World IDP Architecture
38	Scaling the Simple IDP
39	Production IDP Components

1	Analyze the Entire Project First
40	MFA
41	Password Reset
42	Audit Logging
43	Rate Limiting
44	Debugging Guide
45	Testing
46	Complete End-to-End Example
47	Mermaid Diagrams
48	Folder Structure
49	File-by-File Explanation
50	IDP Glossary
51	Common Beginner Confusions
52	Kafka / Microservices Integration
53	IDP vs Custom Login
54	Interview Questions
55	Two-Minute Project Explanation
56	Five-Minute Technical Explanation
57	Learning Path
58	Documentation Files
59	Master Markdown Document
60	Complete PDF
61	PDF Quality Check
62	Final Master Flow
63	Most Important Question
64	Accuracy Rules

Topic 1: Analyze the Entire Project First

In simple words

Documentation must follow source code, not assumptions.

What this project does

The repository contains three Flask services, four HTML templates, two SQLite databases, RSA key files, a runner, and an end-to-end test. The analysis found real OAuth/OIDC, JWT, refresh-token, scope, and session behavior; unimplemented items are called out as recommendations.

Workflow

```
Read source -> list routes -> inspect database schema -> trace requests -> separate
implemented behavior from production advice.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 2: Project Overview

In simple words

An IDP centralizes login and issues proof that other services can trust.

What this project does

The browser starts at the client on port 5005. The client sends the user to the IDP on port 5003; the IDP authenticates the user and issues tokens; the resource server on port 5004 protects profiles and documents.

Workflow

```
User -> Client -> IDP login -> authorization code -> client token exchange -> Resource API.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 3: What Is an Identity Provider?

In simple words

An IDP answers: who is this person?

What this project does

Identity is the user record; authentication proves the user; authorization decides what the user may do. In the building analogy, the IDP is the security gate, the token is an entry pass, and the resource API is a restricted room.

Workflow

```
Credentials -> identity check -> signed pass -> API checks pass and permission.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 4: Why Do We Need an IDP?

In simple words

Without an IDP every application repeats login and password storage.

What this project does

This project demonstrates one login authority serving a client and a separate API. Centralization enables consistent password handling, token issuance, SSO-style redirects, and shared policy, although a production IDP needs stronger operational controls.

Workflow

```
Many app logins -> one IDP -> reusable, audience-specific tokens.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 5: Complete IDP Architecture

In simple words

The architecture separates identity, user experience, and business data.

What this project does

The IDP owns users and token signing. The client owns browser pages and its Flask session. The resource server owns private documents and verifies tokens with the IDP public key rather than receiving the private key.

Workflow

```
Browser -> client:5005 -> IDP:5003; client -> resource:5004; resource -> IDP JWKS when its
key cache expires.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 6: Technology Stack

In simple words

Each component has a small, clear responsibility.

What this project does

Python and Flask provide HTTP services; SQLite stores identity and documents; PyJWT creates and reads JWTs; cryptography handles RSA keys; Werkzeug hashes passwords; requests connects services; Flask-CORS allows cross-origin API calls; HTML templates render the UI.

Workflow

```
Browser HTTP -> Flask route -> Python logic -> SQLite or service-to-service request.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 7: Complete User Authentication Flow

In simple words

Login is a chain of verifiable steps, not one magic action.

What this project does

The normal UI path is OAuth Authorization Code: the client creates state, the IDP checks credentials, the IDP stores a one-time code, the client exchanges it, and the API later verifies the access token.

Workflow

```
Login click -> state -> /authorize -> password check -> code -> /token -> access/id/refresh
tokens -> API call.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 8: User Registration Flow

In simple words

Registration creates an identity record after basic validation.

What this project does

POST /register accepts JSON or form data with username, email, password, and optional full_name. It rejects missing fields, passwords under six characters, and duplicate username/email values, then stores a salted Werkzeug hash.

Workflow

```
Form -> POST /register -> validate -> duplicate check -> hash -> INSERT users -> 201
response.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 9: Password Security

In simple words

The database stores a password hash, never the original password.

What this project does

idp_server.py uses generate_password_hash during registration and check_password_hash during login. The algorithm is delegated to Werkzeug; the code never needs to reverse the hash.

Workflow

```
Password -> salted one-way hash -> database; entered password -> verifier -> match or reject.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 10: Login Flow

In simple words

Login proves a known user controls the password.

What this project does

The direct endpoint POST /login looks up username, verifies the stored hash, and returns a short-lived RS256 access token plus a refresh token. The UI normally uses /authorize instead, so the client never receives the user's password.

Workflow

```
Credentials -> user lookup -> hash verification -> token creation -> JSON response or 401.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 11: Login API Documentation

In simple words

An API contract tells callers exactly what to send and expect.

What this project does

POST /login accepts username and password as JSON or form data. Success returns access_token, token_type, expires_in, refresh_token, and user profile fields; missing fields return 400 and bad credentials return 401.

Workflow

```
POST /login {username,password} -> idp_server.login() ->
create_access_token/create_refresh_token -> 200 JSON.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 12: Token Explanation

In simple words

A token is a portable, time-limited statement issued by a trusted authority.

What this project does

The IDP creates access tokens; the client stores them in its Flask session; the client sends the access token as Authorization: Bearer to the resource server; the resource server verifies it.

Workflow

```
Login -> IDP signs token -> client session -> Bearer header -> API verifies.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 13: JWT Explanation

In simple words

JWT is a signed data format: HEADER.PAYLOAD.SIGNATURE.

What this project does

This project uses RS256 and key id idp-key-1. Access-token claims include iss, sub, aud, iat, exp, user_id, email, name, and scope. JWT payloads are readable; the signature protects integrity but does not encrypt the contents.

Workflow

```
Header names algorithm/key -> payload carries claims -> signature proves IDP signed the exact bytes.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 14: JWT Creation Flow

In simple words

The IDP signs a token only after authentication succeeds.

What this project does

create_access_token builds claims with a 15-minute lifetime and audience resource_server, then PyJWT signs the token with the RSA private key. create_id_token creates the OIDC identity token for the client audience.

Workflow

```
Verified user -> claims -> RS256 private-key signature -> token returned.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 15: JWT Validation Flow

In simple words

The API trusts a token only after several checks pass.

What this project does

require_token parses Bearer syntax, reads kid/alg, loads the matching public key from JWKS, verifies RS256 signature, issuer, audience, and expiration, then checks the required scope.

Workflow

```
Header -> kid -> JWKS public key -> signature/iss/aud/exp -> scope -> request.current_user.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 16: Cookies vs Local Storage vs Memory

In simple words

Where authentication state is stored changes the attack surface.

What this project does

This client uses Flask's session cookie mechanism to hold access_token, refresh_token, id_token, and user data. The code does not use localStorage or sessionStorage. The current app generates a random SECRET_KEY at startup, so sessions are invalidated on restart; production should use a stable secret and secure cookie flags.

Workflow

```
OAuth callback -> server session values -> browser cookie -> client reads session on later
requests.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 17: Authentication vs Authorization

In simple words

Authentication asks who you are; authorization asks what you may do.

What this project does

The IDP authenticates Alice by verifying her password. The resource server authorizes her request by checking scopes such as profile, read, or write.

Workflow

```
Password proof -> identity -> token -> scope check -> allowed or forbidden.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 18: Roles

In simple words

A role is a named bundle of permissions.

What this project does

No roles table or role-checking middleware exists in this project. Access is scope-based, so a role model is a production recommendation rather than an implemented feature.

Workflow

```
Future: user -> role -> permissions -> claims/policy -> endpoint.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 19: Permissions

In simple words

Permissions are the small actions an API allows.

What this project does

The implemented permission-like values are OAuth scopes: openid, profile, email, read, and write. resource_server.py checks profile on /api/profile, read on GET /api/data, and write on POST /api/data.

Workflow

```
Scope claim -> required_scope -> 200 or 403 insufficient_scope.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 20: Protected Route Flow

In simple words

A protected route rejects requests before business logic runs.

What this project does

GET /api/profile is decorated with require_token(required_scope='profile'). Once verified, it returns identity data from the token claims. GET and POST /api/data use read and write scopes and query or update SQLite documents.

Workflow

```
Request -> middleware -> token claims -> scope -> handler -> response.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 21: Complete API Documentation

In simple words

The project exposes public, identity, client, and protected-resource endpoints.

What this project does

IDP: /, /.well-known/openid-configuration, /jwks, /register, /login, /token/refresh, /authorize, /token, /userinfo. Resource: /, /api/health, /api/profile, /api/data GET/POST. Client: /, /login, /callback, /profile, /data GET/POST, /refresh-token, /logout.

Workflow

```
Route -> validation/middleware -> database or remote service -> JSON, HTML, redirect, or
error.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 22: Authentication API Categories

In simple words

Grouping endpoints makes a large API easier to learn.

What this project does

Registration is /register; direct login is /login; OAuth login is /authorize plus /token; refresh is /token/refresh or /token with grant_type refresh_token; identity is /userinfo; protected resources are /api/profile and /api/data. Password reset, role management, and permission management are not implemented.

Workflow

```
Choose category -> call endpoint -> receive identity/token/resource result.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 23: Logout Flow

In simple words

Logout in this demo clears the client session.

What this project does

GET /logout calls session.clear() in client_app.py and redirects home. It does not revoke refresh tokens in the IDP or invalidate already-issued JWTs; a still-valid token may work until expiry.

Workflow

```
Logout click -> local session cleared -> home; production -> revoke tokens and/or end IDP
session.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 24: Refresh Token Flow

In simple words

Refresh tokens let the client obtain a new access token without asking for the password again.

What this project does

The IDP stores opaque refresh tokens in SQLite with username, client_id, scope, expiry, and revoked flag. The client retries a 401 once by posting grant_type=refresh_token to /token; access tokens last 15 minutes and refresh tokens seven days.

Workflow

```
Expired access token -> refresh token -> IDP lookup/expiry/revocation check -> new access
token -> retry.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 25: Session-Based Authentication

In simple words

A session links a browser to server-side application state.

What this project does

The client uses Flask sessions to remember OAuth tokens and profile data between requests. The IDP and resource server do not use login sessions for API authorization; they use tokens.

Workflow

```
Browser cookie -> client Flask session -> token added to outbound API request.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 26: OAuth 2.0

In simple words

OAuth 2.0 lets a client obtain limited access without handling the user's password.

What this project does

This project implements the Authorization Code flow: /authorize authenticates and creates a code, /token exchanges the one-time code for tokens, and the resource server accepts the access token.

Workflow

```
Client redirect -> user authenticates at IDP -> code -> back-channel token exchange -> API.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 27: OpenID Connect

In simple words

OIDC adds identity information on top of OAuth 2.0.

What this project does

The project implements OIDC discovery, an id_token from the code exchange, and /userinfo. OAuth supplies authorization to the resource server; OIDC supplies standardized identity claims to the client.

Workflow

```
OIDC discovery -> authorization code -> id_token/access_token -> /userinfo.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 28: ID Token vs Access Token

In simple words

These tokens have different audiences and jobs.

What this project does

The id_token is created for client_app_demo and describes the authenticated user. The access token is created for audience resource_server and is sent to protected APIs. The API must not use an ID token as an API credential.

Workflow

```
ID token -> client identity; access token -> resource API authorization.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 29: Complete Request to Response Flow

In simple words

Every operation can be followed from user action to response.

What this project does

A login button triggers a redirect; an authorization form triggers credential verification; a callback exchanges a code; a profile page calls the API; middleware validates the JWT; the API returns claims.

Workflow

```
Frontend event -> HTTP request -> Flask route -> helper/database -> response -> next request.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 30: Error Handling

In simple words

Errors tell the caller whether to fix input, authenticate, or change permissions.

What this project does

The code uses 400 for malformed requests, 401 for missing/invalid credentials or tokens, 403 for missing scopes, 404 for missing users, 409 for duplicate registration, and 201/200 for successful writes/reads. Database and network failures are logged but not consistently mapped to a production error model.

Workflow

```
Failure -> status code + JSON/HTML message -> client redirect, retry, or display.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 31: Security Analysis

In simple words

The project demonstrates important controls but is educational, not production-ready.

What this project does

Implemented: password hashing, RS256 signatures, issuer/audience/expiry checks, state checking, one-time authorization codes, refresh expiry/revocation fields, parameterized SQLite queries, and scopes. Gaps: HTTP URLs, permissive CORS, hard-coded client secret, key files in the repo, no rate limiting, no MFA, no audit log, and no strict redirect validation at the token endpoint.

Workflow

```
Control -> threat reduced; missing control -> production work item.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 32: Environment Variables

In simple words

Configuration should be separated from source code.

What this project does

The resource and client services read IDP_BASE_URL and the client reads RESOURCE_SERVER_URL. The IDP issuer, client ID/secret, redirect URI, lifetimes, and paths are code constants; secrets should move to environment variables or a secret manager.

Workflow

```
Environment/config -> service startup -> routes use trusted values.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 33: Database Architecture

In simple words

The identity database stores identity protocol state; the resource database stores business data.

What this project does

idp_database.db has users, clients, auth_codes, and refresh_tokens. resource_database.db has user_documents. There are no roles, permissions, sessions, audit_logs, or separate credentials tables.

Workflow

```
users -> codes/tokens; token subject -> user_documents.username.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 34: Database Flow

In simple words

Database writes and reads are tied to authentication stages.

What this project does

Registration inserts a hashed user. Login reads the user and creates a refresh-token row. Authorization inserts a one-time code. Token exchange marks that code used and creates tokens. Resource access reads or inserts documents for the token subject.

Workflow

```
Validate -> parameterized SQL -> commit -> JSON/HTML result.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 35: IDP vs Application

In simple words

The IDP proves identity; the application delivers business features.

What this project does

Here the IDP owns login, users, codes, keys, and tokens. The client owns pages and session handling. The resource server owns document data and business authorization based on scopes.

Workflow

```
IDP issues proof -> application presents proof -> resource server serves business data.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 36: Single Sign-On

In simple words

SSO means one IDP login can serve multiple client applications.

What this project does

The current demo has one registered client, but its authorization-code design is the foundation for SSO. Additional clients would need registered IDs, validated redirect URIs, and an IDP browser session to avoid repeated login.

Workflow

```
App A -> IDP session -> App B/App C receive their own audience-specific tokens.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 37: Real-World IDP Architecture

In simple words

Production identity platforms separate edge, identity, keys, sessions, policy, and observability.

What this project does

This demo compresses those responsibilities into one Flask IDP process and SQLite. A production system commonly adds an API gateway, durable database, key rotation/JWKS publishing, session and token services, audit logs, MFA, and monitoring.

Workflow

```
Clients -> gateway -> identity services -> databases/keys/events.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 38: Scaling the Simple IDP

In simple words

Scaling means removing single-process and single-file bottlenecks.

What this project does

At small scale this local SQLite design is easy to understand. At higher scale use a managed relational database, indexes, multiple stateless service instances, shared session/refresh infrastructure, cache, key rotation, rate limiting, and telemetry.

Workflow

```
One process -> replicated services -> shared durable state -> monitored capacity.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 39: Production IDP Components

In simple words

Each production component addresses a failure or security concern.

What this project does

Useful additions include a load balancer, gateway, identity/token services, Redis for short-lived state, a durable database, audit logging, monitoring, rate limiter, secret manager, KMS/HSM, notifications, and MFA service.

Workflow

```
Request -> edge controls -> identity policy -> durable state -> signed result -> audit event.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 40: MFA

In simple words

Multi-factor authentication combines different kinds of proof.

What this project does

MFA is not implemented. A production flow could require password plus authenticator-app OTP, WebAuthn/passkey, or a carefully evaluated notification factor; SMS/email should be treated as weaker recovery channels.

Workflow

```
Password -> second factor -> authentication success -> token.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 41: Password Reset

In simple words

Password reset must prove control of a recovery channel without exposing the old password.

What this project does

Reset is not implemented. A safe future flow creates a short-lived, single-use, hashed reset token, sends a link, verifies it, updates the password hash, expires sessions/tokens, and records the event.

Workflow

```
Forgot password -> generic response -> one-time email link -> new hash -> revoke old auth
state.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 42: Audit Logging

In simple words

Audit logs answer who did what, when, and from where.

What this project does

No audit-log table or event pipeline is implemented. Production should record successful and failed logins, logout, refresh, password changes, role changes, token revocation, and administrative actions without storing secrets.

Workflow

```
Security event -> structured log -> protected retention/search -> alert or investigation.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 43: Rate Limiting

In simple words

Authentication endpoints are attractive targets for brute-force attacks.

What this project does

No rate limiter or account lockout is implemented. Production should throttle by account and network signals, add monitoring and progressive delays, and consider CAPTCHA only where appropriate.

Workflow

```
Repeated attempts -> limiter -> delay/block/challenge -> alert.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 44: Debugging Guide

In simple words

Debug from the outside inward, following one request at a time.

What this project does

For login, check redirect URL, state, form fields, IDP logs, user row, password verifier, and token response. For 401, check the Bearer header, kid/alg, JWKS, issuer, audience, expiry, and refresh path. For 403, inspect the required scope.

Workflow

```
Browser -> client log -> IDP log -> resource log -> response status.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 45: Testing

In simple words

Tests turn an authentication flow into repeatable evidence.

What this project does

test_flow.py exercises discovery, JWKS, registration, direct login, refresh, authorization code, replay prevention, UserInfo, protected profile/data, missing token, and tampered-token rejection. Password reset, MFA, role policy, and logout revocation tests are absent because those features are absent.

Workflow

```
Start services -> run test_flow.py -> assert status, claims, and security behavior.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 46: Complete End-to-End Example

In simple words

The most important story is login followed by a protected API request.

What this project does

The browser uses client_app.login, the IDP's authorize route verifies credentials and creates a code, client_app.callback calls /token, and client_app.profile calls resource /api/profile. The resource middleware validates the token and scope before returning the profile.

Workflow

```
Click login -> /authorize -> code -> /callback -> /token -> session -> /api/profile -> 200.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 47: Mermaid Diagrams

In simple words

Diagrams make request ownership and trust boundaries visible.

What this project does

The guide includes text workflows suitable for Mermaid: architecture, registration, login, token creation/validation, protected API, logout, SSO, database relationships, and production architecture. Every diagram should keep the three ports and trust boundaries explicit.

Workflow

```
Sequence participants -> request arrows -> validation -> response arrows.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 48: Folder Structure

In simple words

Files are organized by service and responsibility.

What this project does

Top-level Python files are the three services plus key generation, runner, and tests. templates contains home, login, profile, and data pages. Two SQLite files hold separate domains; PEM files hold signing material.

Workflow

```
Service file -> route/helper -> template or database.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 49: File-by-File Explanation

In simple words

A useful code reading method connects each file to its caller and security responsibility.

What this project does

idp_server.py owns identity protocol logic; resource_server.py owns JWKS validation and documents; client_app.py owns redirects, sessions, and API calls; generate_keys.py creates RSA keys; run_all.py starts processes; test_flow.py verifies behavior; templates render user-facing pages.

Workflow

```
Entry point -> route -> helper -> external dependency -> output.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 50: IDP Glossary

In simple words

A glossary gives beginners a stable vocabulary.

What this project does

Identity means account information; authentication means proving identity; authorization means permission; access token means API credential; refresh token means token-renewal credential; JWT means signed claims format; OAuth means delegated authorization; OIDC means identity on OAuth; RBAC means role-based access control.

Workflow

```
Learn term -> map it to a route/claim/table -> use it in explanations.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 51: Common Beginner Confusions

In simple words

Similar-sounding security terms solve different problems.

What this project does

JWT is a format, not authentication by itself. JWTs are signed rather than encrypted here. OAuth is not JWT; OIDC is not identical to OAuth; an IDP is a service, not merely a database; logout clears this client session but does not revoke JWTs.

Workflow

```
Question -> identify protocol/data format/storage -> explain its exact project role.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 52: Kafka / Microservices Integration

In simple words

Events can notify other services without coupling them to login requests.

What this project does

Kafka is not present. A future user-created event could feed notification, analytics, or provisioning services, but identity creation should succeed or fail independently from noncritical consumers.

Workflow

```
Registration -> committed user -> event -> consumers -> retries/dead-letter handling.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 53: IDP vs Custom Login

In simple words

Custom login is one application feature; an IDP is a reusable trust service.

What this project does

This repository is a learning implementation of an IDP, not a replacement for hardened products in high-risk production. Dedicated IAM is valuable when many apps, SSO, federation, MFA, lifecycle, compliance, or delegated access are required.

Workflow

```
Single app need -> local auth; many apps/security requirements -> shared IDP.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 54: Interview Questions

In simple words

Interview preparation is strongest when answers trace this code.

What this project does

Prepare beginner, intermediate, and advanced questions around routes, password hashing, RS256/JWKS, code replay, scopes, sessions, refresh, OAuth/OIDC, 401 vs 403, key rotation, scaling, and missing controls. The generated topic file contains a compact question bank prompt and project anchors.

Workflow

```
Question -> identify exact file/function -> explain tradeoff -> name production improvement.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 55: Two-Minute Project Explanation

In simple words

A short explanation should name the trust boundaries and main flow.

What this project does

Say: this is a Flask IDP ecosystem with a client and resource server. The client redirects to the IDP, receives an authorization code, exchanges it for tokens, stores them in a session, and calls the API with a Bearer JWT. The API verifies RS256 using JWKS and enforces scopes.

Workflow

```
Problem -> architecture -> login -> token -> protected API -> security caveat.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 56: Five-Minute Technical Explanation

In simple words

A deeper explanation adds claims, databases, errors, and tradeoffs.

What this project does

Walk through browser, client state, IDP users and codes, RSA signing, token lifetimes, callback exchange, JWKS cache, issuer/audience checks, scope enforcement, SQLite documents, refresh, logout, and production gaps.

Workflow

```
User -> client -> IDP -> authN -> code/tokens -> API authN -> authZ -> resource.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 57: Learning Path

In simple words

Learn the system from transport to production policy.

What this project does

Level 1 is HTTP/APIs; then authentication; password hashing; JWT and sessions; scopes/RBAC; OAuth; OIDC; SSO; MFA; and production IAM. Each level should be practiced against a route, claim, or test in this repository.

Workflow

```
Concept -> small experiment -> observe request/response -> add security reasoning.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 58: Documentation Files

In simple words

Documentation should be modular enough to read one concept at a time.

What this project does

This deliverable creates docs/topics/01 through 64 as individual Markdown pages, plus a master Markdown document and a complete PDF. The project README is updated with links so the guide is discoverable.

Workflow

```
Topic file -> master index -> PDF page -> repository README link.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 59: Master Markdown Document

In simple words

The master guide is the single searchable source of truth.

What this project does

COMPLETE_IDP_DOCUMENTATION.md combines all 64 topic pages, a project facts summary, endpoint inventory, architecture, workflows, security analysis, glossary, interview guidance, and final flow. It preserves the distinction between implemented features and recommendations.

Workflow

```
Repository evidence -> topic pages -> consolidated guide.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 60: Complete PDF

In simple words

The PDF is a printable version with one numbered topic per page.

What this project does

COMPLETE_IDP_DOCUMENTATION.pdf includes a title page, contents page, project facts, readable workflows, endpoint examples, diagrams in text form, security notes, glossary, and interview guidance. Each of the 64 numbered topics starts on its own page.

Workflow

```
Markdown content -> styled report -> page breaks -> numbered PDF pages.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 61: PDF Quality Check

In simple words

A document is not complete until its rendering is inspected.

What this project does

The generated PDF uses fixed margins, page breaks, readable fonts, page numbers, and compact tables. Validation checks page count, extracted topic headings, presence of page numbering, and rendered page images for clipping or overlap.

Workflow

```
Generate -> render PNGs -> inspect representative pages -> extract text -> final check.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 62: Final Master Flow

In simple words

The complete flow joins browser, IDP, tokens, API checks, and resource output.

What this project does

The browser reaches the client, the client starts login, the IDP authenticates and consults SQLite, signs a token, and the client calls the resource API. The resource server validates signature, issuer, audience, expiry, and scope before returning documents.

Workflow

```
User -> client -> login -> IDP -> authN/database -> token -> API -> authN -> authZ ->
resource.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 63: Most Important Question

In simple words

A confident answer traces what, why, where, input, output, failure, and production handling.

What this project does

For this project: username/password enter the IDP authorization form; idp_server.authorize checks the user and stores a code; client_app.callback exchanges it at /token; the client stores tokens in session; resource_server.require_token validates the access token; the endpoint applies a scope and returns the resource.

Workflow

```
Input -> route -> helper -> storage/crypto -> output; failures become 400/401/403/409 or
redirect messages.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.

Topic 64: Accuracy Rules

In simple words

The documentation must never turn a recommendation into a fact.

What this project does

This guide is grounded in the inspected files and live SQLite schemas. It explicitly says that roles, password reset, MFA, audit logging, rate limiting, and Kafka are not implemented, and it avoids reproducing secret values or real token strings.

Workflow

```
Inspect source -> verify behavior -> document exact file/function -> label gaps -> protect
secrets.
```

Project anchor

This repository is a three-service Flask demonstration: idp_server.py (IDP, port 5003), resource_server.py (protected API, port 5004), and client_app.py (web client, port 5005).

Implemented vs production

Implemented here: SQLite users/clients/authorization codes/refresh tokens, Werkzeug password hashing, OAuth 2.0 Authorization Code flow, OIDC discovery and UserInfo, RS256 JWTs, JWKS verification, scopes, refresh, client sessions, and protected documents.

Production improvement: move secrets and keys to a secret manager, use HTTPS, tighten cookie settings and CORS, validate redirect URIs strictly, add rate limiting and audit logs, and use a mature IAM product or hardened service design.